

Тестовое задание на стажировку DevOps

Теоретическая часть

1. Чем git pull отличается от git fetch?

Ответ: git fetch только скачивает актуальную информацию из удалённого репозитория и обновляет локальное представление удалённых веток. При этом текущая локальная ветка сама по себе не изменяется.

git pull сначала получает изменения с удалённого репозитория, а затем сразу пытается объединить их с текущей локальной веткой. То есть git pull - это git fetch + последующее объединение изменений. В зависимости от настроек это может быть fast-forward, merge или rebase.

2. Чем в Linux отличается soft link от hard link? Какое поведение будет при удалении файла оригинала? Можно ли создать hardlink на директорию?

Ответ: Символическая ссылка - это по сути ярлык, который хранит путь до другого файла или каталога. Путь может быть как абсолютным, так и относительным. Если удалить оригинальный файл, символическая ссылка останется, но станет битой и больше не сможет привести к файлу. Символические ссылки можно создавать на каталоги и на объекты, находящиеся в другой файловой системе.

Жёсткая ссылка - это ещё одно имя для того же inode и тех же данных. Поэтому нельзя в обычном смысле сказать, что один из файлов является оригиналом, а другой ссылкой - они равноправны. Если удалить одно имя файла, данные останутся на диске, пока существует хотя бы одна hard link на этот inode. Hard link нельзя создать между разными файловыми системами, потому что inode имеет смысл только внутри конкретной файловой системы. Обычно hard link на директории создавать нельзя. Это ограничение нужно для того, чтобы не появлялись циклы в дереве каталогов и не ломалась структура файловой системы.

3. Как проверить сетевую доступность между двумя Linux машинами?

Ответ: Можно проверить через ping. Маршрут до узла можно посмотреть через traceroute или mtr. Если нужно проверить конкретный TCP-порт, можно использовать nc. Для HTTP/HTTPS использовать curl. Если ping не отвечает, это ещё не означает, что машина недоступна: ICMP может быть заблокирован.

4. На каких компонентах Linux основана контейнеризация в Docker?

Ответ: На namespaces и cgroups.

namespaces отвечают за изоляцию. Определяют, какие процессы, сеть, точки монтирования и другие ресурсы видит контейнер. cgroups отвечают за учёт и ограничение ресурсов, например CPU и памяти. Контейнер использует ядро хостовой Linux-системы, но процессы при этом изолированы.

5. Почему вместо "COPY . ." / "RUN npm install" рекомендуют делать "COPY package.json" / "RUN npm install" / "COPY . ."?

Ответ: Это связано с кешированием слоёв Docker.

Docker кеширует шаги сборки, в том числе COPY и RUN. Если сначала скопировать весь проект, то изменение исходного кода изменит слой COPY. Из-за этого кеш для следующего шага станет неактуальным и npm install придётся выполнять заново.

Если сначала скопировать package.json и package-lock.json, если он есть, установить зависимости, а уже потом копировать исходный код, то при неизменных файлах зависимостей и предыдущих шагах Docker сможет взять результат установки из кеша.

Изменение обычного исходного кода не заставит Docker заново устанавливать все зависимости.

6. Могут ли два контейнера внутри одного Pod в Kubernetes слушать один и тот же порт?

Ответ: Нет, если речь идёт об одном и том же IP-адресе и одном и том же протоколе.

Контейнеры внутри одного Pod используют общий network namespace, поэтому у них один IP-адрес и общее пространство сетевых портов. Если один контейнер уже слушает 0.0.0.0:8080, второй контейнер в этом же Pod не сможет занять этот же адрес и порт. При этом они могут слушать разные порты, например один 8080, а другой 9090.

7. Какие виды JOIN знаете и чем они отличаются?

Ответ: INNER JOIN возвращает только те строки, для которых нашлось совпадение в обеих таблицах.

LEFT JOIN возвращает все строки из левой таблицы и подходящие строки из правой. Если совпадения нет, значения правой таблицы будут NULL.

RIGHT JOIN работает наоборот: возвращает все строки из правой таблицы и совпадения из левой.

FULL JOIN возвращает все строки из обеих таблиц: как совпавшие, так и те, для которых пары не нашлось.

CROSS JOIN возвращает все возможные комбинации строк из двух таблиц.

8. Что такое HAVING в SQL запросе? Чем отличается от WHERE?

Ответ: WHERE фильтрует отдельные строки до группировки. HAVING фильтрует уже сформированные группы после GROUP BY. В HAVING можно использовать агрегатные функции, например HAVING COUNT(*) > 1.

Практическая часть

Репозиторий: <https://github.com/y1g0ul/hello-minikube>

Docker Hub: <https://hub.docker.com/r/y1g0ul/hello-minikube>

Приложение написано на Flask, слушает порт 32777 и возвращает ASCII Hello World. Для него подготовлен Dockerfile, образ опубликован в Docker Hub.

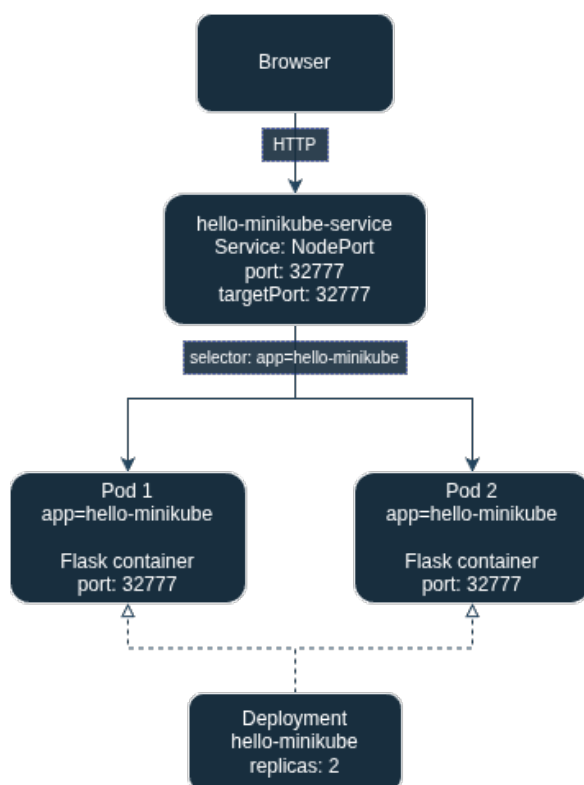
В Minikube приложение развёрнуто через Deployment с двумя репликами. Для доступа к Pod используется Service типа NodePort. Манифесты Deployment и Service находятся в каталоге k8s/.

Minikube работает на отдельной VM. Для доступа через браузер с основного компьютера я использовал SSH-туннель и `kubectl port-forward`:

```
ssh -L 32777:127.0.0.1:32777 user@192.168.1.83 \  
'kubectl port-forward service/hello-minikube-service 32777:32777'
```

После этого открыл `http://127.0.0.1:32777` в браузере.

Схема архитектуры



Исходник схемы draw.io: docs/architecture.drawio в репозитории.

Результаты работы

Скриншот 1. Pod

```
y1g0ul@fedora:~$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
hello-minikube-5595d995c8-4gpx6	1/1	Running	1 (120m ago)	129m
hello-minikube-5595d995c8-mhnkd	1/1	Running	1 (120m ago)	129m

Скриншот 2. Deployment и Service

```
y1g0ul@fedora:~$ kubectl get deployments
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
hello-minikube	2/2	2	2	132m

```
y1g0ul@fedora:~$ kubectl get services
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
hello-minikube-service	NodePort	10.109.201.35	<none>	32777:32564/TCP	128m
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	143m

Скриншот 3. Ответ приложения через NodePort

```
y1g0ul@fedora:~$ minikube service hello-minikube-service --url
http://192.168.49.2:32564
y1g0ul@fedora:~$ curl http://192.168.49.2:32564
```

```
$$\  $$\      $$\  $$\
$$ |  $$ |      $$ | $$ |
$$ |  $$ |  $$$$$$\  $$ | $$ |  $$$$$$\  $$\  $$\  $$$$$$\  $$ |  $$$$$$\  $$ |
$$$$$$$$$ |$$  _$$\  $$ | $$ |  $$  _$$\  $$ | $$ |  _$$\  $$ |
$$  _$$ |$$$$$$$$$ | $$ | $$ |  $$ /  $$ |  _$$ | $$ | /  $$ |
$$ |  $$ | $$      _$$ | $$ | $$ |  $$ |  $$ |  _$$ | $$ |  _$$ |
$$ |  $$ | \$$$$$$$\  $$ | $$ | \$$$$$$$\  $$ | \$$$$$$$\  $$ | \$$$$$$$\  $$ |
\__|  \__|  \_____|  \__| \__|  \_____|  \__| \_____|  \__|  \_____|  \__|
```

Скриншот 4. Ответ приложения в браузере через проброс портов

The screenshot shows a web browser window with the address bar displaying '127.0.0.1:32777'. The browser content area displays the same ASCII art output as seen in the previous screenshot, rendered in a monospaced font.

CI/CD

Дополнительно я настроил CI/CD через GitHub Actions. При push в main с изменением приложения или файлов, влияющих на сборку, GitHub Actions собирает новый Docker image и отправляет его в Docker Hub. Для Pull Request образ только собирается, без публикации и деплоя.

После успешной сборки отдельный deploy job запускается на self-hosted runner на VM с Minikube и применяет манифесты Service и Deployment. В Deployment подставляется тег вида sha-<commit SHA>, поэтому запущенную версию можно связать с конкретным commit.

Если изменились только манифесты в k8s/, сборка образа пропускается. Deploy job применяет манифесты с тем image, который уже используется в кластере. Если Deployment ещё не создан, используется latest из манифеста. Каталог k8s/ исключён из Docker-контекста через .dockerignore.

Изменения README и файлов документации новую сборку и деплой при обычном push в main не запускают. Для первого запуска в манифесте указан latest с imagePullPolicy: Always. Последующие изменения применяются через CI/CD с SHA-тегом. Ручной kubectl apply -f k8s/ вернёт latest; обновление этого тега само по себе не перезапускает работающие Pod.

Скриншот 5. Успешный запуск CI/CD

Triggered via push 1 hour ago

Status: Success

Total duration: 54s

Artifacts: 1

docker.yml
on: push

docker (35s) → deploy (14s)

docker summary

Docker Build summary

For a detailed look at the build, download the following build record archive and import it into Docker Desktop's Builds view. Build records include details such as timing, dependencies, results, logs, traces, and other information about a build. [Learn more](#)

y1g0ul-hello-minikube-3MF1ZR.dockerbuild (44.94 KB - includes 1 build record)

Find this useful? [Let us know](#)

ID	Name	Status	Cached	Duration
3MF1ZR	hello-minikube	completed	0%	13s

► Build inputs

Job summary generated at run-time

Artifacts
Produced during runtime

Name	Size	Digest
y1g0ul-hello-minikube-3MF1ZR.dockerbuild	44.9 KB	sha256:dc3b8bf28b6bfbb63be678cf507c5fd65a728824c539d1cfcddc79f4633c383b

Запуск на скриншоте: <https://github.com/y1g0ul/hello-minikube/actions/runs/35219216568>